Kubernetes in Action

Chapter 1: 쿠버네티스 소개

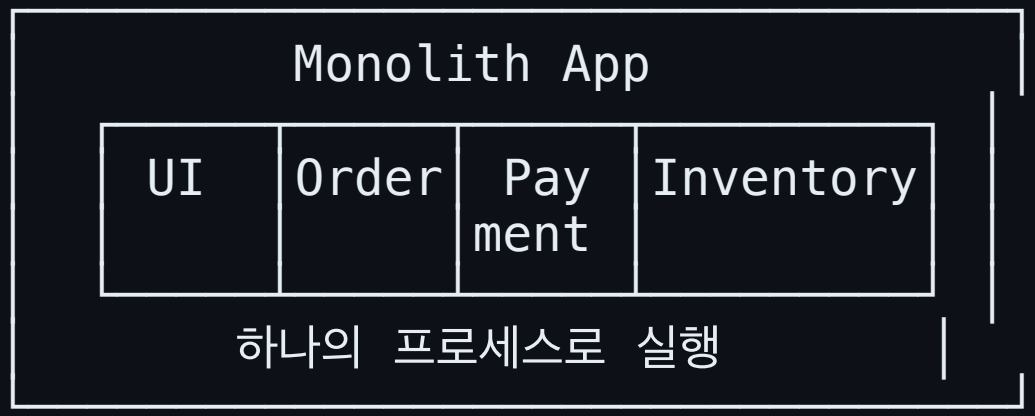
Kubernetes in Action 1st Edition — Marko Lukša

목차

1. **쿠버네티스가 필요한 이유** — Monolith → Microservices
2. **컨테이너 기술** — VM vs Container, Docker
3. **쿠버네티스 아키텍처** — Control Plane & Worker Node
4. **애플리케이션 실행 흐름** — 선언적 모델
5. **Traditional MSA vs Kubernetes** — 무엇이 바뀌었는가
6. **요약**

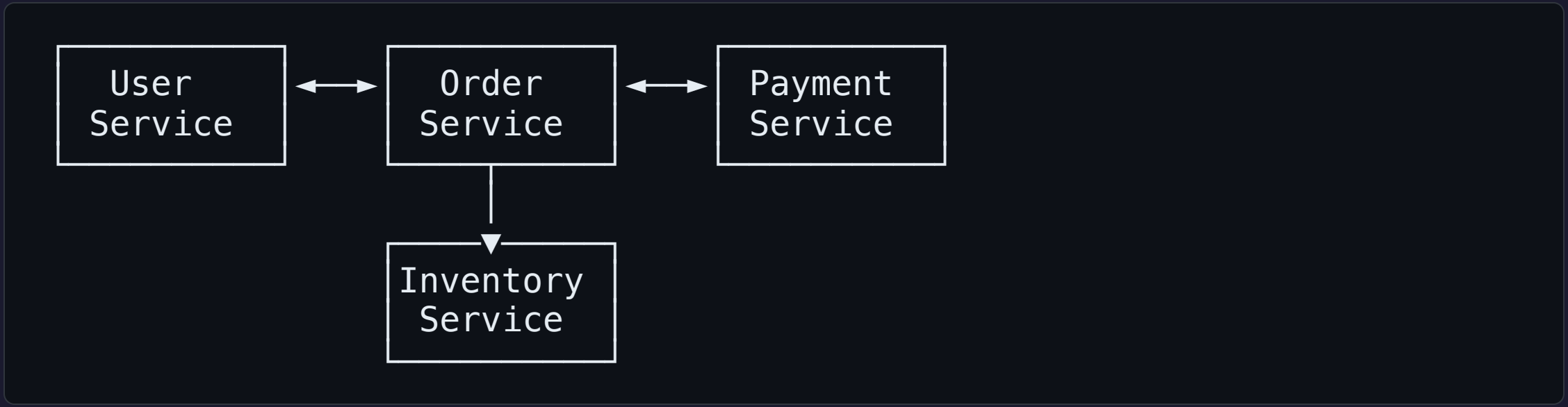
1. 왜 쿠버네티스가 필요한가?

Monolith의 한계



- 모든 컴포넌트가 **하나의 단위**로 개발, 배포, 스케일링
- **수직 스케일링**: 더 큰 서버 (비용↑, 물리적 한계)
- **수평 스케일링**: 전체를 복제 (리소스 낭비)
- 하나의 버그가 **전체 시스템**을 다운시킬 수 있음

Microservices로의 전환



장점: 개별 개발, 배포, 스케일링 가능

새로운 문제:

- 🔍 서비스끼리 **서로를 어떻게 찾는가?** (Service Discovery)
- 🔗 **내부 통신**은 어떻게 하는가?
- ⚙️ 서로 다른 **라이브러리 버전** 충돌

DevOps → NoOps

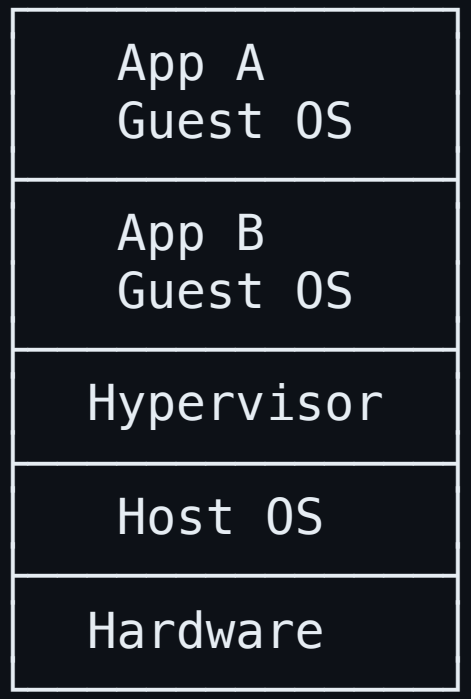
- DevOps: 개발 팀이 배포 + 운영까지 책임
- NoOps: 인프라가 완전히 추상화 → 개발자는 앱만 제출

> 쿠버네티스가 이 NoOps를 가능하게 한다.
> 개발자는 앱을 제출하고, K8s가 어디서 실행할지 결정한다.

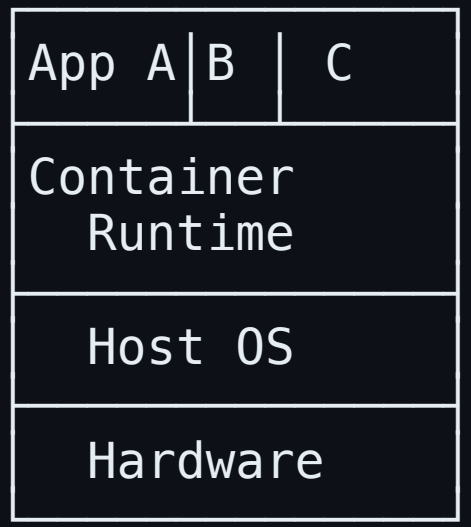
2. 컨테이너 기술

VM vs Container

VM 방식



Container 방식



VM vs Container — 비교표

	가상머신	컨테이너
격리	완벽 (별도 커널)	프로세스 수준 (커널 공유)
오버헤드	GB 단위 RAM	MB 단위
시작 시간	분 단위	**초 단위**
밀도	호스트당 수~십 개	호스트당 **수백 개**
보안	강함	커널 공유 리스크
이식성	이미지 크기 큼	레이어 공유로 **효율적**

컨테이너 격리 메커니즘

Linux Namespaces — "무엇을 볼 수 있는가"

각 컨테이너에 **독립된 시스템 뷰** 제공

Namespace	격리 대상
Mount	파일시스템
PID	프로세스 ID
Network	네트워크 인터페이스
IPC	프로세스 간 통신
UTS	호스트명
User	사용자 ID

Linux cgroups — "얼마나 쓸 수 있는가"

CPU, 메모리, 디스크 I/O, 네트워크 **사용량 제한**

Docker

Build → Ship → Run

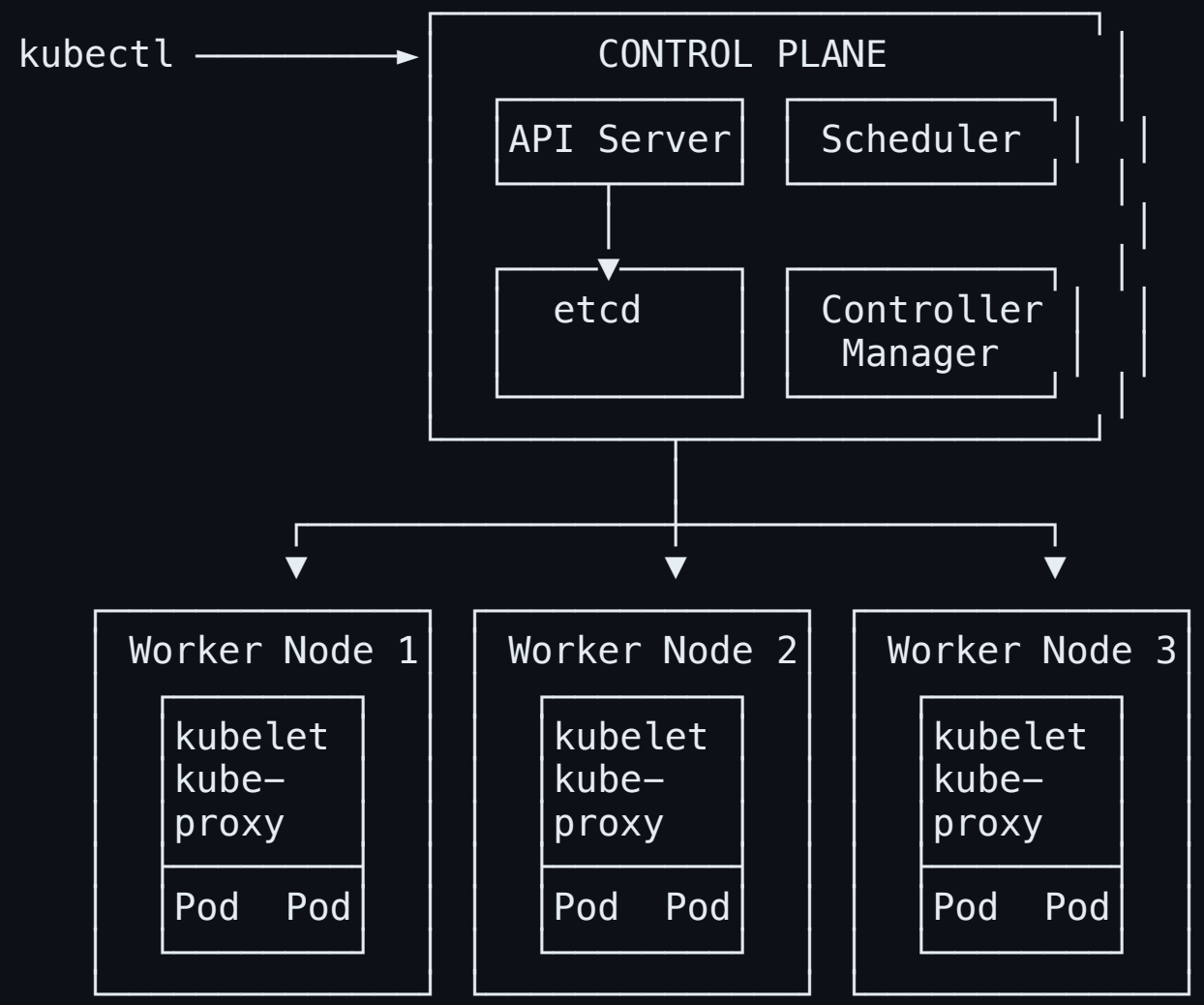
```
Developer          Registry          Production
┌──────────┐      ┌──────────┐      ┌──────────┐
│docker    │──────│Docker    │──────│docker    │
│build     │ push │Hub       │ pull │run       │
└──────────┘      └──────────┘      └──────────┘
```

- **이미지**: 앱 + 환경을 레이어로 패키징
- **레지스트리**: 이미지 저장/배포 (Docker Hub)
- **컨테이너**: 이미지의 실행 인스턴스
- **레이어 공유**: 여러 이미지가 동일 레이어 재사용 → 저장/전송 효율

> Docker는 격리를 제공하는 것이 아니라,
> 커널의 **Namespace + cgroups**를 쉽게 쓸 수 있게 해주는 플랫폼이다.

3. 쿠버네티스 아키텍처

전체 구조


kubectl
CONTROL PLANE
API Server
Scheduler
etcd
Controller Manager
Worker Node 1
kubelet
kube-proxy
Pod
Pod
Worker Node 2
kubelet
kube-proxy
Pod
Pod
Worker Node 3
kubelet
kube-proxy
Pod
Pod

Control Plane 구성요소

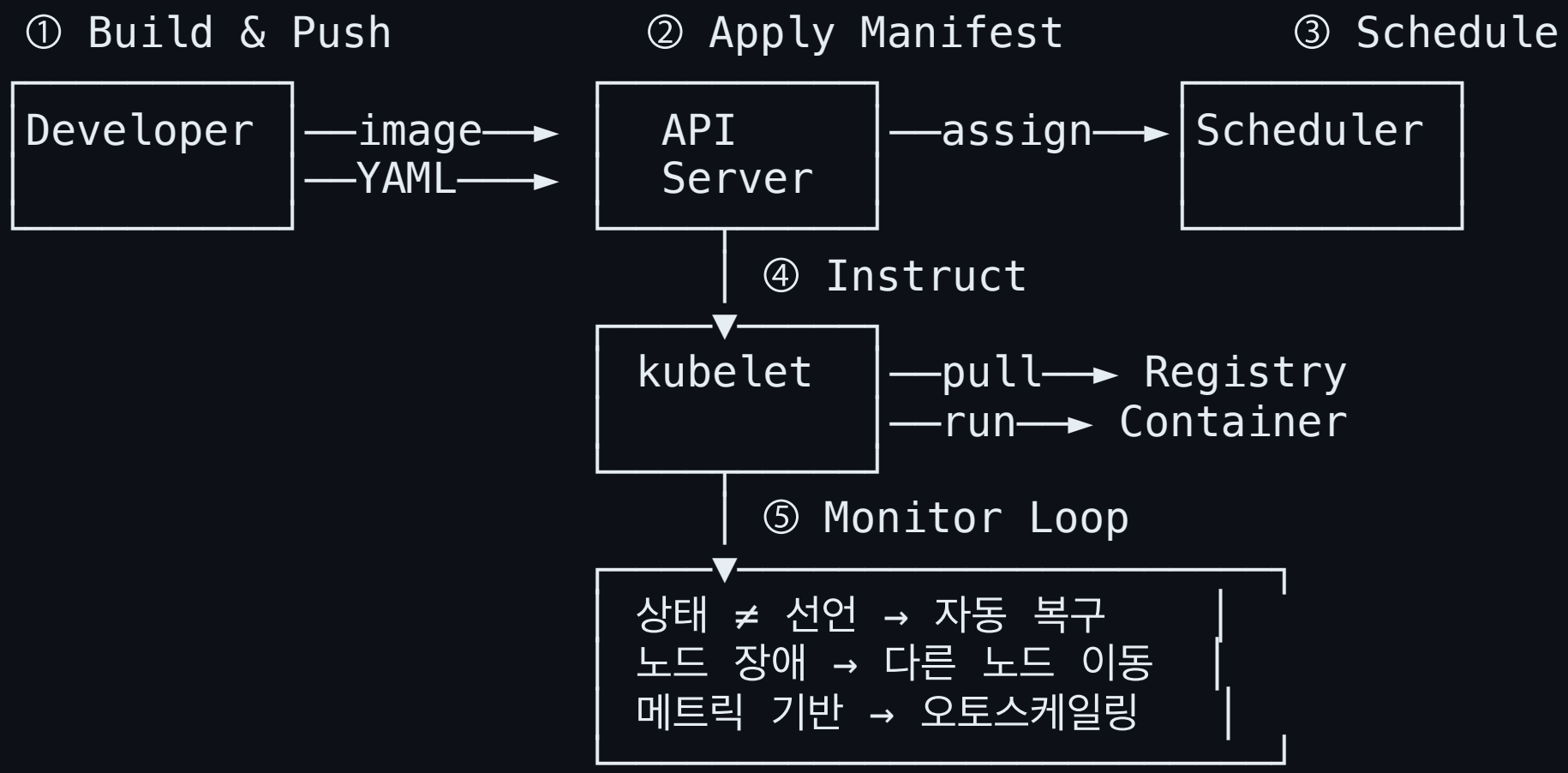
구성요소	역할
API Server	모든 통신의 **중심 허브** — kubectl, kubelet 모두 여기로
etcd	클러스터 전체 상태를 저장하는 **분산 KV 스토어**
Scheduler	리소스 요구사항에 따라 **Pod를 노드에 배치**
Controller Manager	복제본 관리, 노드 추적, **장애 자동 처리**

Worker Node 구성요소

구성요소	역할
kubelet	API Server와 통신, 해당 노드의 **컨테이너 관리**
kube-proxy	Service의 네트워크 트래픽 **로드밸런싱** (iptables/IPVS)

4. 애플리케이션 실행 흐름

배포 프로세스


① Build & Push
② Apply Manifest
③ Schedule
Developer
image
YAML
API Server
assign
Scheduler
④ Instruct
kubelet
pull
Registry
run
Container
⑤ Monitor Loop
상태 ≠ 선언 → 자동 복구
노드 장애 → 다른 노드 이동
메트릭 기반 → 오토스케일링

선언적 모델 (Declarative)

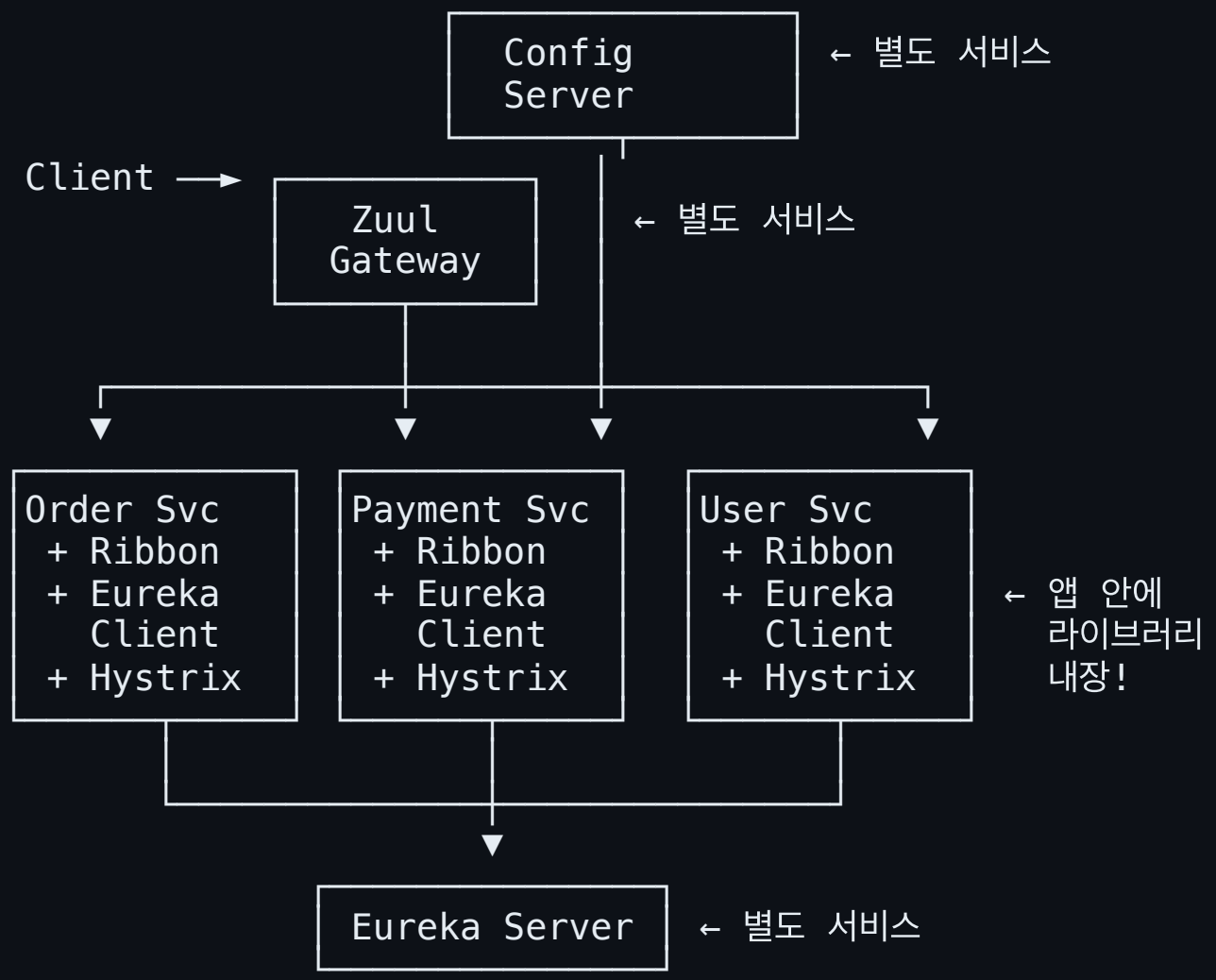
> "**이 상태를 유지해라**"라고 선언하면, K8s가 알아서 맞춘다.

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: order-service
spec:
  replicas: 3                    # ← 항상 3개 유지
  template:
    spec:
      containers:
      - name: order
        image: order-service:1.0
```

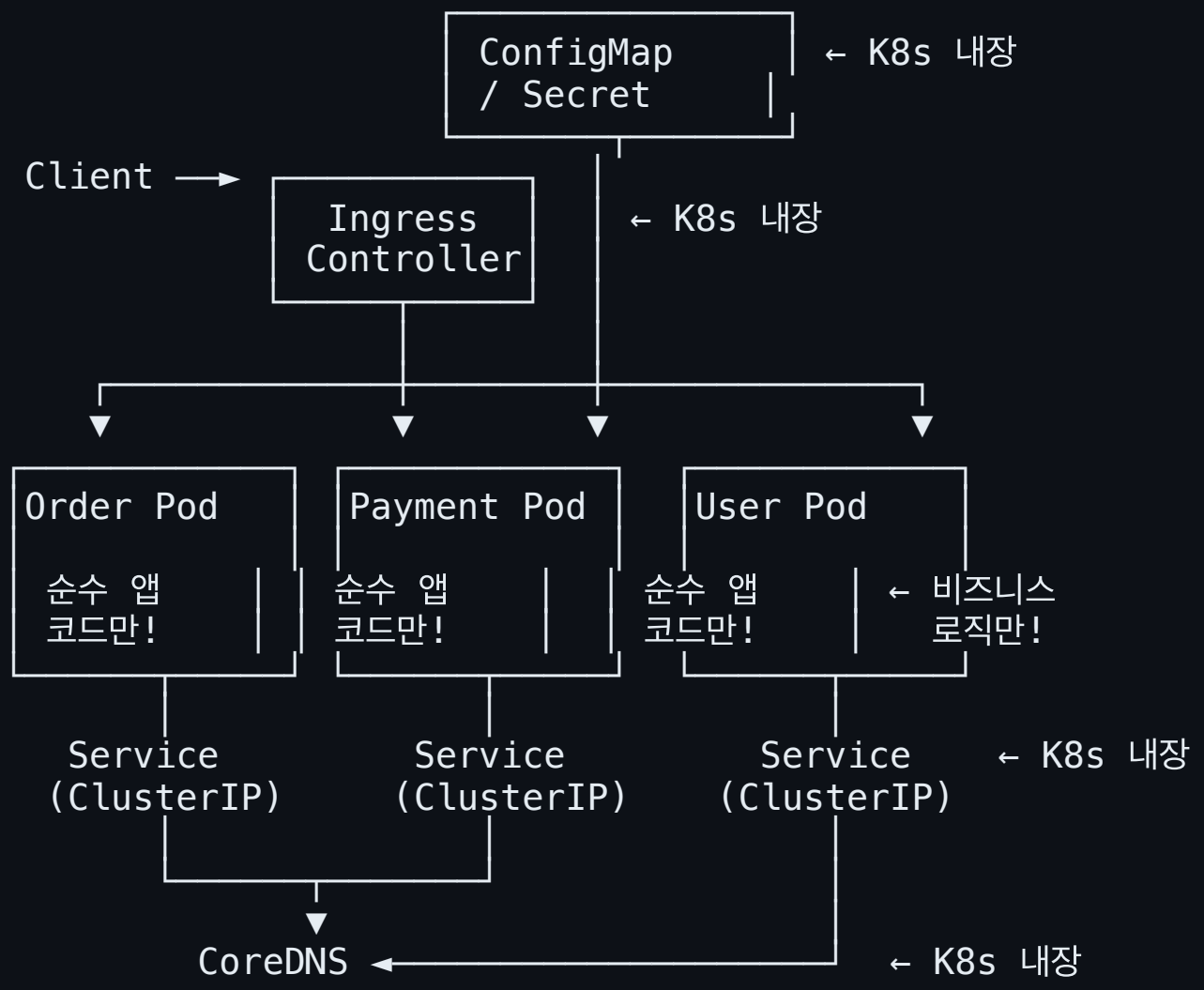
- Pod 죽으면 → **자동 재생성** / 노드 다운 → **다른 노드에서 재시작**
- CPU 임계치 초과 → **오토스케일링**

5. Traditional MSA vs Kubernetes

기존 MSA 아키텍처 (Spring Cloud Netflix)



Kubernetes MSA 아키텍처



서비스 디스커버리 — Before: Eureka

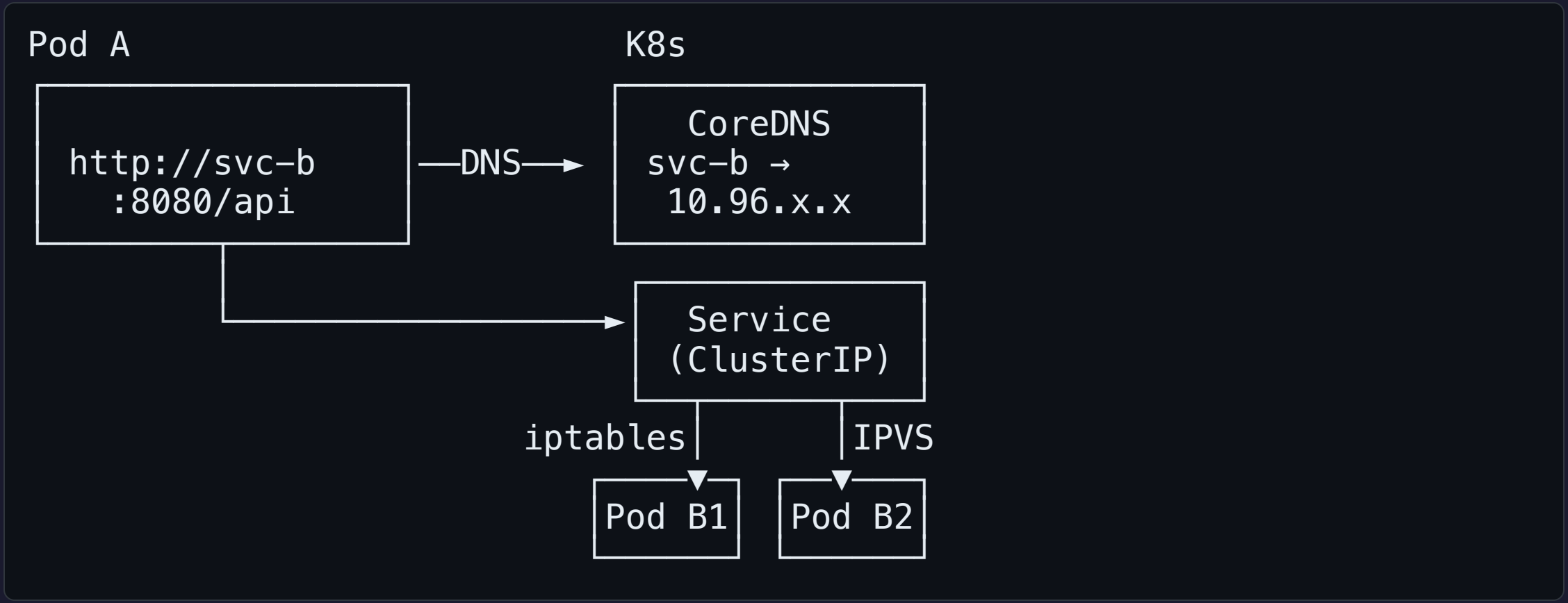
```
Service A                          Eureka Server

 EurekaClient        ──reg──►     서비스 목록
 Ribbon              ◄─list──
 DiscoveryClient                   30s 주기
    .getInstances                  heartbeat

          │ client-side LB
          ▼
      Service B (IP 직접 호출)
```

- 모든 서비스에 **Eureka Client + Ribbon** 내장 (JVM 종속)
- Eureka Server **별도 운영** 필요
- heartbeat 누락 시 **최대 90초** stale 엔트리 유지

서비스 디스커버리 — After: K8s Service + CoreDNS



- 앱 코드에 SDK 불필요 — **DNS 이름만 사용**
- Endpoint Controller → Pod 변경 **즉시 갱신** / 컨트롤 플레인 **내장**

서비스 디스커버리 — 비교

	Eureka + Ribbon	K8s Service + CoreDNS
레지스트리	별도 서버 운영 필요	**컨트롤 플레인 내장**
언어 지원	JVM 중심	**언어 무관 (DNS)**
Stale 엔트리	최대 **90초**	**즉시 갱신**
앱 코드 결합	`DiscoveryClient` 필요	**제로 — DNS만 사용**
장애 시	Registry 다운 = 디스커버리 불가	**etcd 기반 HA**

코드 변화:

```
discoveryClient.getInstances("order") → http://order-service:8080
```

내부 통신 & 로드밸런싱

	Ribbon (in-process)	kube-proxy (kernel)
LB 위치	앱 프로세스 **내부**	커널 레벨 (**iptables/IPVS**)
캐시	Eureka 전파 지연 → stale	API Server **직접 watch**
성능	앱 스레드 사용	**O(1) IPVS** 조회
L7 라우팅	Zuul (thread-per-request)	Ingress + **Istio**

설정 관리

	Spring Cloud Config	ConfigMap + Secret
별도 서비스	Config Server 운영 필요	**불필요**
언어	Spring 전용	**언어 무관**
변경 반영	Cloud Bus + RabbitMQ	Reloader or Rolling Restart
감사	커스텀 로깅	**Git 히스토리 (GitOps)**
장애 시	Config Server 다운 = 앱 시작 불가	etcd 기반

상태 확인 & 자가 치유

	Actuator + Eureka	K8s Probes
감지 → 조치	수동 모니터링 + 스크립트	**kubelet 자동 재시작**
감지 시간	~90초 (heartbeat 3회)	**5~10초**
부분 장애	구분 없음	**Liveness vs Readiness**

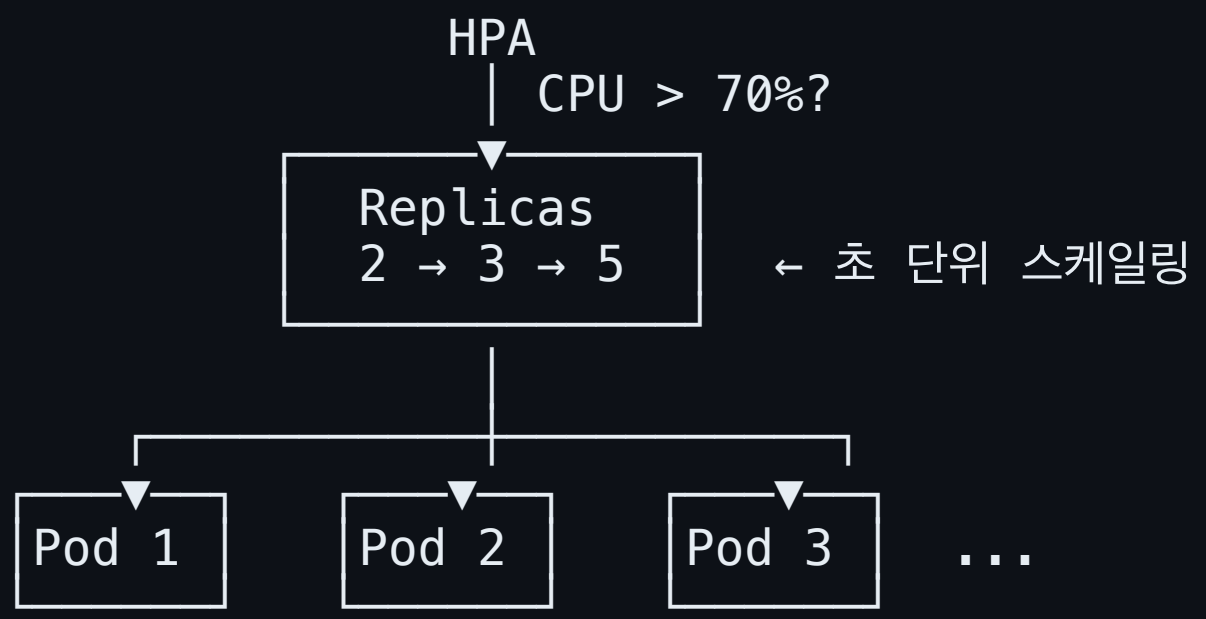
K8s Probe 구분

```
Liveness Probe 실패                Readiness Probe 실패
        |                                  |
        ▼                                  ▼
  컨테이너 재시작                  Service에서 제거 (재시작 X)
  (프로세스 죽음)                   (일시적 장애: DB 연결 끊김 등)
```

핵심: Readiness 실패 = 트래픽만 제거, 재시작 안 함
DB 일시 장애 시 불필요한 재시작 방지

스케일링

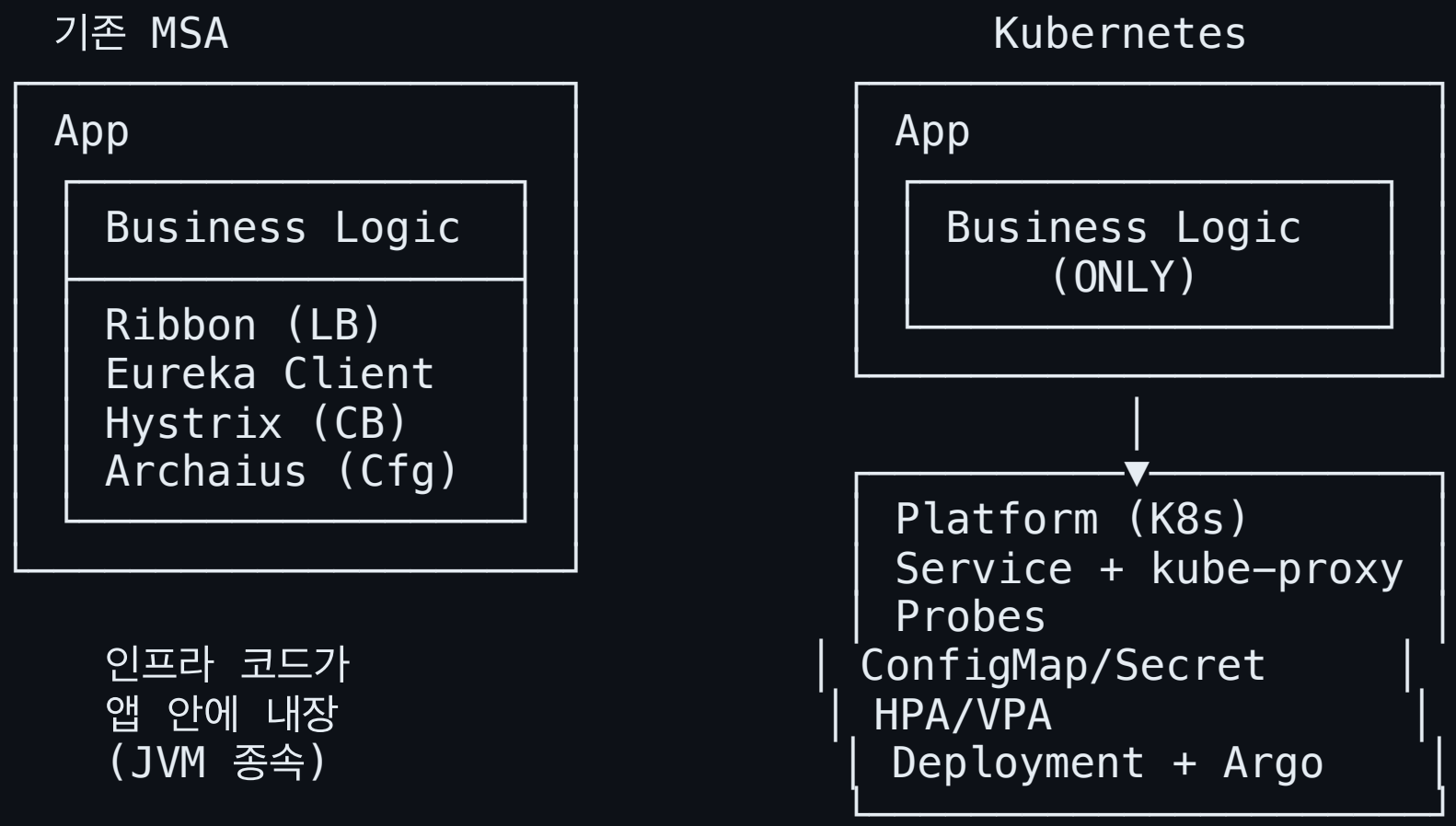
	VM 기반 (ASG)	K8s HPA
단위	VM (분 단위)	Pod (초 단위)
트리거	CPU만	CPU, 메모리, 커스텀 메트릭
Scale-to-zero	불가	KEDA로 가능
세분성	VM 전체	서비스(Pod) 단위



배포 전략

	Spinnaker/Jenkins	K8s + Argo
Rolling Update	커스텀 파이프라인	**내장, 선언적**
운영 비용	Spinnaker **8+ 서비스**	Argo = **CRD 1개**
선언적	명령형 파이프라인	**YAML in Git**
Drift 감지	없음	**ArgoCD 지속 동기화**
Rollback	수동	`kubectl rollout undo`

핵심 변화: 인프라 관심사의 이동



요약

Chapter 1 핵심 정리

1. **Monolith → Microservices**: 독립 배포 가능하지만, 운영 복잡도 급증
2. **Container**: VM보다 가볍게 격리 제공 (Namespace + cgroups)
3. **Docker**: Build → Ship → Run 표준화
4. **Kubernetes**: 데이터센터를 **하나의 컴퓨팅 리소스**로 추상화
5. **선언적 모델**: "이 상태를 유지해라" → K8s가 자동으로

> 기존 MSA는 인프라 관심사를 **앱 라이브러리에 내장** (JVM 종속)
> K8s는 이를 **플랫폼 계층으로 이동** (언어 무관)

Before vs After — 한눈에 보기

관심사	Before (앱 내부)	After (K8s 플랫폼)
Discovery	Eureka Client	CoreDNS + Service
LB	Ribbon	kube-proxy (IPVS)
Config	Config Server	ConfigMap/Secret
Health	Actuator + 수동	Liveness/Readiness
Scaling	ASG (분)	HPA/KEDA (초)
Deploy	Spinnaker (8 svc)	Deployment + Argo

Q&A

Thank you!